

Amérique du Nord – 2025 – sujet2 - Correction

Exercice 2 (6 points)

PARTIE A

Question 1

```
pegula = Joueuse("Pegula", "Jessica", "USA", 29, 6101)
```

👉 *Commentaire : On instancie un objet de la classe `Joueur` en appelant le constructeur avec les informations demandées.*

Question 2

```
def ajouter_victoire(self, adversaire):  
    self.victoire = self.victoire + 1  
    adversaire.defaite = adversaire.defaite + 1
```

👉 *Commentaire : La joueuse courante gagne une victoire tandis que son adversaire reçoit une défaite.*

Question 3

```
swiatek.ajouter_victoire(paloni)
```

👉 *Commentaire : C'est Swiatek qui gagne le match, donc on appelle la méthode sur l'objet `swiatek`.*

Question 4

Le coût temporel du tri par insertion dans le pire des cas est quadratique: $O(n^2)$

👉 *Commentaire : Dans le pire des cas, chaque élément doit être comparé à tous les précédents. On peut aussi dire que cet algorithme effectue une double boucle.*

Question 5

Liste initiale :

```
[swiatek, gauff, paloni, sabalenka, pegula]
```

Classement par points WTA croissants :

- paloni : 4843
- gauff : 6063
- pegula : 6101
- swiatek : 7470
- sabalenka : 10541

Tableau complété :

Étape	Contenu de <code>liste_joueuses</code>	Joueuses échangées
0	[swiatek, gauff, paloni, sabalenka, pegula]	-
1	[gauff, swiatek, paloni, sabalenka, pegula]	swiatek, gauff
2	[gauff, paloni, swiatek, sabalenka, pegula]	swiatek, paloni
3	[paloni, gauff, swiatek, sabalenka, pegula]	gauff, paloni
4	[paloni, gauff, swiatek, pegula, sabalenka]	sabalenka, pegula
5	[paloni, gauff, pegula, swiatek, sabalenka]	swiatek, pegula

👉 *Commentaire : le tri par insertion place progressivement chaque joueuse à la bonne position.*

Question 6

```
def resultat_match(self, score):
    self.score = score
    nb_set_joueuse1 = 0
    nb_set_joueuse2 = 0
    for set_match in score:
        if set_match[0] > set_match[1]:
            nb_set_joueuse1 = nb_set_joueuse1 + 1
        else:
            nb_set_joueuse2 = nb_set_joueuse2 + 1
    if nb_set_joueuse1 > nb_set_joueuse2:
        self.gagnante = self.joueuse1
        self.perdante = self.joueuse2
    else:
        self.gagnante = self.joueuse2
        self.perdante = self.joueuse1
    self.gagnante.ajouter_victoire(self.perdante)
```

👉 Commentaires :

- On compte le nombre de sets gagnés par chaque joueuse afin de déterminer la gagnante du match.
 - Une fois la gagnante connue, on met à jour les statistiques grâce à la méthode `ajouter_victoire`.
-

PARTIE B

Question 7

Un tournoi de tennis peut être modélisé par un arbre binaire car chaque match possède deux matchs précédents dont les gagnantes se rencontrent au tour suivant.

👉 *Commentaire : Chaque nœud de l'arbre représente un match et possède deux sous-arbres correspondant aux matchs précédents.*

Question 8

```
tournoi = Arbre(  
    F,  
    Arbre(  
        D1,  
        Arbre(Q1, None, None),  
        Arbre(Q2, None, None)  
    ),  
    Arbre(  
        D2,  
        Arbre(Q3, None, None),  
        Arbre(Q4, None, None)  
    )  
)
```

👉 *Commentaire : La racine représente la finale, puis chaque sous-arbre représente les demi-finales et les quarts.*

Question 9

```
tournoi.gauche.racine.joueuse1 = gauff
```

👉 *Commentaire : `tournoi.gauche` correspond à la première demi-finale D1.*

Question 10

Un programme récursif est un programme qui s'appelle lui-même.

👉 *Commentaire : Une fonction récursive résout souvent un problème en le ramenant à des sous-problèmes plus petits.*

Question 11

```
1 def mise_a_jour(self):
2     """Met à jour les matchs de l'arbre"""
3     if self.racine.joueuse1 is None:
4         if self.gauche is not None:
5             # mise à jour si gagnante à gauche
6             if self.gauche.racine.gagnante is not None:
7                 self.racine.joueuse1 = self.gauche.racine.gagnante
8             else:
9                 self.gauche.mise_a_jour()
10    if self.racine.joueuse2 is None:
11        if self.droit is not None:
12            # mise à jour si gagnante à droite
13            if self.droit.racine.gagnante is not None:
14                self.racine.joueuse2 = self.droit.racine.gagnante
15            else:
16                self.droit.mise_a_jour()
```

👉 *Commentaires :*

*ligne 6 : on vérifie si le match du sous-arbre gauche possède déjà une gagnante.
ligne 7 : la gagnante du match de gauche devient joueuse1 du match courant.
ligne 9 : sinon, on continue récursivement la mise à jour dans le sous-arbre gauche.
ligne 13 : on vérifie si le match du sous-arbre droit possède déjà une gagnante.
ligne 14 : la gagnante du match de droite devient joueuse2 du match courant.
ligne 16 : sinon, on poursuit récursivement la mise à jour dans le sous-arbre droit.*
